

Computer Organization & Assembly Language (EE2003)

Sessional-I Exam

Total Time (Hrs): 1
Total Marks: 45
Total Questions: 3

Date: September 23rd 2025

Course Instructor(s)

Dr. Aatira Anum Ahmad, Ms. Samin Iftikhar

Ms. Aleena Ahmad, Ms. Sana Fatima

Mr. Zummar Saad

Roll No

Section

Student Signature

Do not write below this line

Instruction/Notes: This is an open book exam. Only textbook by "Belal Hashmi" is allowed. Sharing book or calculators is NOT ALLOWED. PAPERS/NOTES ATTACHED WITH BOOK ARE NOT ALLOWED. Rough sheets can be used but will not be collected and checked. In case of any ambiguity, make reasonable assumptions. Questions during exams are not allowed.

CLO # 1: Demonstrate the basic concepts of computer organization including CPU, memories, and input/output and explain their purposes and interactions.

Q1: Short Questions [1+ 4 + 2 + 5 marks]

(a) What is the maximum memory accessed (in GB) by a computer system if it has an address bus of 40 bits? $2^{40} = 2^{10} \times 2^{10} \times 2^{10} = 2^{10} \text{ GB}$

(b) (i) Write a single instruction that toggles 3 least significant bits of BL register. XOR BL, 0x07

(ii) Write a single instruction that flips even positions (bits 0, 2, 4, ...) in AL register. XOR AL, 0xAA/0x55

(iii) Write two instructions that checks if the 4th position bit of AL register is set without changing the destination register. $\text{TEST AL, 0x0000000b}$ JZ 81/JZ 81

(c) For each of the instructions given below, identify whether the instruction is valid or invalid.

Instruction	Valid/Invalid	Instruction	Valid/Invalid
mov [bx-2], ax	valid	mov [mem1], al	valid
mov [bx+si-2], 10h	invalid	mov word[dx], 3	invalid

(d) Given that AX is initialized with 0x8888, what are the values of the flags after execution of the following code. Also tell whether the jump will be taken or not. Show your working to get credit.

Fall 2025

FAST School of Computing

Page 1 of 4

1000 1000 1000 1000
0111 0111 0111 0111
0001 0001 0001 0001

Instructions	ZF	CF	SF	OF	Jump Taken	Working
cmp ax, 0x7777 jg ll	0	0	0	1	Not Taken	0988 7777

CLO # 2: Describe the working of important x86 assembly primitives, including arithmetic, branching, bit manipulation, addressing modes and interrupt handling.

Q2: [3+8+2 marks]

(a) Content of the memory at the logical address [0x92F5:0x1234] can be read at [0x8ABC:0x95C4]. Show your working to get credit.

92F50 1234 ----- 94184	94184 8ABC0 ----- 95C4	(1.5)
---------------------------------	---------------------------------	-------

(b) What will be the content of memory after the execution of following code?

<pre>[org 0x0100] mov ax, 0xABCD mov [num1+2], ax mov ax, 0xEF12 mov [num3+3], ax mov ax, 0x4c00 int 0x21 num1: db 5, 10, 15 num2: dw 14, 15 num3: dd 0x3004, 0x4c00</pre>	
---	--

Memory Configuration after Execution:

Memory Address	num1+0	+1	+2	+3	+4	+5	+6	+7	+8
Memory Content	05	0A	CD ^{0F}	AB ⁰⁰	00 ⁰⁰	0F	00	04	30
Memory Address	+9	+A	+B	+C	+D	+E	+F		
Memory Content	00	12 ⁰⁰	EF ⁰⁰	4C	00	00			

(c) After execution of code given below, what are the values of registers AX and BX in hexadecimal?

```
[org 0x0100]
mov ax, 0x6BCD
mov bx, 0x1234
rol ax, 3
rci bx, 1
```

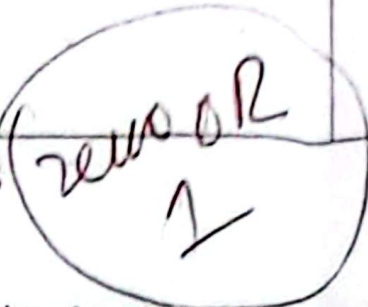
Show your working here.

ax:

0x5E6B

bx:

0x2469



CLO # 3: Apply the knowledge of Intel x86 architecture to develop moderately complex and well-modularized assembly programs.

Q3: [20 marks] Given an array, write an assembly program to count the number of elements in that array that have at least 3 consecutive 1's in it. E.g., if you are given the following array

array1: db 0x12, 0x1F, 0x46, 0xE3, 0x97

Number in Hex	Number in Binary	At least 3 consecutive 1's
0x12	0001 0010	No
0x1F	0001 1111	Yes
0x46	0100 0110	No
0xE3	1110 0011	Yes
0x97	1001 0111	Yes

As shown in the table, the above array has 3 elements that have at least 3 consecutive 1's in it.

You should store the number in a data label named 'count'. Your program should work for any array. Assume that the array size is given in a data label named 'size'.

```
[org 0x100]
jmp start
array1: db 0x12, 0x1F, 0x46, 0xE3, 0x97
size: db 5
count: db 0
```

start: